TF2 in Practice

Publishing frames, asking for transforms, and reading the four errors

Prof. Anis Koubaa Dr. Asem Ibrahim Alalwan

EE 414 — Introduction to Robotics
College of Engineering
Alfaisal University

Fall 2026 — Week 7, Lecture B

How this session works

You will not multiply a single matrix today. TF2 does the algebra.

Your job is the part TF2 cannot do for you: **publish the right links**, and **ask the right question**. Both are easy to get subtly wrong, and both fail loudly — which is good news.

1. Looking at a tree that already exists
2. Publishing a frame
3. Asking for a transform
4. The four errors you will meet

Outline

Part 1

Looking at a tree that already exists

1. Looking at a tree that already exists
2. Publishing a frame
3. Asking for a transform
4. The four errors you will meet

Start the robot, then look at its frames

```
$ ros2 launch turtlebot3_gazebo empty_world.launch.py

$ ros2 run tf2_tools view_frames
Listening to tf data for 5.0 seconds...
Generating graph in frames.pdf
```

Open `frames.pdf`. You are looking at the tree from Lecture A, drawn from the live robot — with the publisher of each link named on the edge.

Checkpoint 1. Everyone has a `frames.pdf` with `odom`, `base_link` and `base_scan` in it.

Ask for one transform, from the terminal

```
$ ros2 run tf2_ros tf2_echo base_link base_scan
At time 0.0
- Translation: [0.000, 0.000, 0.172]
- Rotation: in Quaternion [0.000, 0.000, 0.000, 1.000]
- Rotation: in RPY (radian) [0.000, -0.000, 0.000]
```

The laser is 17.2 cm above `base_link`, unrotated, constant. The dynamic one:

```
$ ros2 run tf2_ros tf2_echo odom base_link
```

Drive with teleop and watch this one change. That is the difference between a static link and a dynamic one, on your screen.

Who publishes what — and why it matters

Link	Published by	Character
`map` → `odom`	AMCL or SLAM (Week 10)	Jumps. Corrects the drift below it.
`odom` → `base_link`	The odometry source	Smooth, continuous, drifts forever
`base_link` → `base_scan`	`robot_state_publisher`, from the URDF (Week 8)	Constant

Why localization inserts a frame

A planner needs *smooth* pose: `odom` → `base_link`. A goal needs *correct* pose: `map` → `base_link`. Splitting them lets one be smooth and the other be right. This is the design you will rely on in Weeks 10 and 11.

Outline

Part 2

Publishing a frame

1. Looking at a tree that already exists
2. Publishing a frame
3. Asking for a transform
4. The four errors you will meet

A static link, with no code at all

Mount a camera 10 cm forward and 20 cm up from the robot centre:

```
$ ros2 run tf2_ros static_transform_publisher \
      --x 0.10 --y 0.0 --z 0.20 \
      --roll 0 --pitch 0 --yaw 0 \
      --frame-id base_link --child-frame-id camera_link
```

```
$ ros2 run tf2_ros tf2_echo base_link camera_link
```

Use the named flags, not bare numbers. The deprecated positional form orders its arguments differently — the source of many backwards-mounted sensors.

Checkpoint 2. `camera_link` appears in `view_frames`.

A dynamic link, from a node

```python
from tf2_ros import TransformBroadcaster
from geometry_msgs.msg import TransformStamped

self.br = TransformBroadcaster(self)

def tick(self):
    t = TransformStamped()
    t.header.stamp = self.get_clock().now().to_msg()
    t.header.frame_id = 'base_link'        <- PARENT
    t.child_frame_id  = 'tool_link'        <- CHILD
    t.transform.translation.x = 0.3
    t.transform.rotation.w = 1.0
    self.br.sendTransform(t)
```

`header.frame_id` is the **parent**, `child_frame_id` the child. Swapping them builds a tree that is inside out — and it will still run.

Two traps in that code

The trap	What it costs you
`rotation.w` left at its default `0.0`	An all-zero quaternion is not a rotation. Every lookup through this link is garbage, and nothing warns you. **Identity is $w = 1$.**
Using `time.time()` instead of `get_clock()`	In simulation the clock is not wall time. Your transform is stamped in a different era from every other message, and lookups fail.

Checkpoint 3. Your `tool_link` appears, and `tf2_echo` shows 0.3 m forward.

Try it once with `w = 0.0` and look at what `tf2_echo` reports. Ten seconds, and you will never forget it.

Outline

Part 3

Asking for a transform

1. Looking at a tree that already exists
2. Publishing a frame
3. Asking for a transform
4. The four errors you will meet

The listener

```
from tf2_ros import Buffer, TransformListener

self.buffer = Buffer()
self.listener = TransformListener(self.buffer, self)

def tick(self):
    try:
        tf = self.buffer.lookup_transform(
                'odom',          <- TARGET: what frame I want the answer in
                'base_scan',     <- SOURCE: what frame the data is in now
                rclpy.time.Time())
    except TransformException as e:
        self.get_logger().warn(f'no transform yet: {e}')
        return
```

`rclpy.time.Time()` is time zero, which means **"the latest you have"**.

The argument order, which everyone gets wrong once

`lookup_transform(target, source, time)`
returns $T_{\text{target, source}}$ — **takes source data *to* target**

Lecture A's notation, unchanged: the inner indices cancel. If you have a point in `base_scan` and want it in `odom`, you ask for `lookup_transform('odom', 'base_scan', t)`.

Why swapping them is so dangerous

The reversed call is **perfectly valid** — it returns the inverse. No error, no warning. Your obstacle simply appears on the wrong side of the robot, and the robot steers into it. This is the frame bug promised in Lecture A.

Transforming actual data

```python
from tf2_geometry_msgs import do_transform_point
from geometry_msgs.msg import PointStamped

p = PointStamped()
p.header.frame_id = 'base_scan'
p.point.x = 2.1

tf = self.buffer.lookup_transform('odom', 'base_scan', rclpy.time.Time())
p_odom = do_transform_point(p, tf)
```

Do not extract the numbers and do the arithmetic yourself. The helper handles the quaternion correctly, and hand-rolled versions are where sign errors live.

Checkpoint 4. Drive the robot; watch a fixed `base_scan` point move in `odom`.

Why that checkpoint is the whole week

A point at 2.1 m in front of the *laser* is a different place in the world every time the robot moves. A point in `odom` stays put.

Obstacle avoidance (Wk 6)	Works in `base_link`: "is something in front of *me*?"
Mapping (Wk 10)	Needs `odom` or `map`: a wall must stay where it is when the robot moves
Navigation (Wk 11)	Needs `map`: the goal does not move when the robot does

Every one of those is the same call with different frame names. That is why this week is worth ninety minutes of your semester.

Outline

Part 4

The four errors you will meet

1. Looking at a tree that already exists
2. Publishing a frame
3. Asking for a transform
4. The four errors you will meet

Cause each one. Read it. Fix it.

Do this	The message	The cause
Look up immediately at start-up	"`base_scan` passed to lookupTransform does not exist"	The buffer is empty. Nothing is wrong — you asked too early.
Ask for `now()` instead of `Time()`	"requires extrapolation into the future"	The transform for that instant has not arrived yet.
Sleep 30 s, then ask for an old stamp	"requires extrapolation into the past"	The buffer discarded it. Your node was too slow.
Publish `odom` → `base_link` from a second node	No error. The robot twitches.	Two parents for one child. **The dangerous one.**

Checkpoint 5. Everyone has seen all four.

The first two are not bugs. Handle them.

```
try:
    tf = self.buffer.lookup_transform('odom', 'base_scan',
                                      rclpy.time.Time())
except TransformException:
    return          <- normal for the first second. Try again next tick.
```

A node that starts before the transform publisher **must** tolerate a few seconds of failure. Nodes start in any order — that is Week 2's no-master design, and this is the price of it.

In the field

Never crash on a missing transform, and never retry in a loop inside the callback. Return, and let the next tick try again. A callback never waits — Week 3's rule, still holding.

The fourth one, found in one command

```
$ ros2 run tf2_tools view_frames
$ ros2 topic info /tf --verbose
Publisher count: 2        <- both publishing odom -> base_link
```

Two publishers of the same link produce a transform that flickers between two answers at whatever rate they happen to run. The robot twitches, the map smears, and **nothing reports an error**.

Why a tree makes this findable

Lecture A: one parent per frame. Because that rule exists, "this child has two parents" is a detectable, meaningful fault. In a general graph it would just be another edge.

Seeing it instead of reading it

In RViz2, add the **TF** display and set the fixed frame to `odom`.

Axes at every frame	Red x, green y, blue z — your sensor's real pose on the robot
Drive with teleop	`base_link` moves; `base_scan` moves rigidly with it
Set fixed frame to `base_link`	Now the *world* moves and the robot sits still. Same data, different question.

In the field

Changing the fixed frame is the fastest way to understand frames, and the fastest way to diagnose one. A sensor mounted backwards is obvious in one glance here, and invisible in a table of numbers.

Before you leave

Leaving this room: a tree you have drawn, a static frame published from the command line, a dynamic frame published from your own node, a point transformed between frames while the robot drives, and all four TF errors caused and read.

`view_frames`	What is the tree, and who publishes each link?
`tf2_echo`	What is this one transform, right now?
RViz2 TF display	Does it look right?

Next week

Week 8 stops publishing frames by hand. You describe the robot once in URDF, and `robot_state_publisher` emits the whole static tree for you — correctly, and from a single source of truth.